

Data transformation

Marcin Włodarczak

21 November 2024

As usual, we start by loading tidyverse:

```
library('tidyverse')
theme_set(theme_bw())
```

We will also need some helper code from 06-utils.R:

```
source('06-utils.R')
```

Simple and multiple regression

So far, we have talked about simple regression, that is models with a single predictor:

```
d1 <- tibble(x = runif(50, -5, 5),
             y = 5 + -9 * x + rnorm(50, 0, 1))
summary(lm(y ~ x, d1))
```

```
##
## Call:
## lm(formula = y ~ x, data = d1)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.61584 -0.80148  0.02571  0.62448  2.94122
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  4.81481    0.15169   31.74  <2e-16 ***
## x           -8.97127    0.05175  -173.37  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.061 on 48 degrees of freedom
## Multiple R-squared:  0.9984, Adjusted R-squared:  0.9984
## F-statistic: 3.006e+04 on 1 and 48 DF,  p-value: < 2.2e-16
```

Sadly, life is rarely so simple that a phenomenon can be adequately explained by a single variable. Usually, we need several predictors to account for a meaningful portion of variance in the data.

Here is an example where y is a function of two variables, x1 and x2, plus intercept and some normally distributed noise:

```
d2 <- tibble(x1 = runif(20, 13, 27),
             x2 = runif(length(x1), -3, 11),
             y = -2 + -6 * x1 + 7 * x2 + rnorm(length(x1), 0, 1))
```

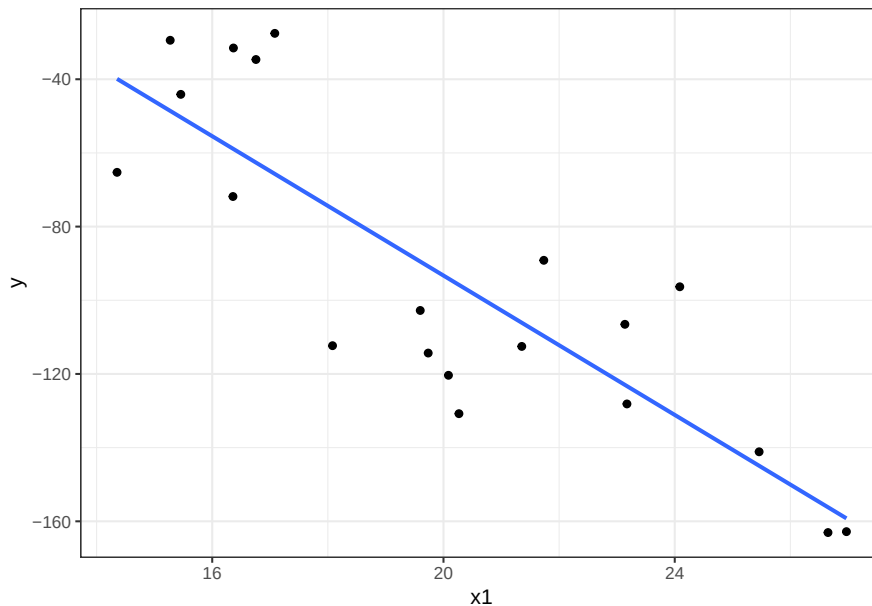
We could of course analyse the relationship between y and x1:

```
m1 <- lm(y ~ x1, d2)
summary(m1)
```

```
##
## Call:
## lm(formula = y ~ x1, data = d2)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -37.183 -15.845  -4.223  19.515  38.153
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   95.858     28.867   3.321  0.0038 **
## x1            -9.457       1.411  -6.701 2.77e-06 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 23.96 on 18 degrees of freedom
## Multiple R-squared:  0.7139, Adjusted R-squared:  0.698
## F-statistic: 44.91 on 1 and 18 DF,  p-value: 2.769e-06
```

```
ggplot(d2, aes(x1, y)) +
  geom_point() +
  geom_smooth(method = 'lm', se = FALSE)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



Similarly, we could analyse the relationship between y and x2:

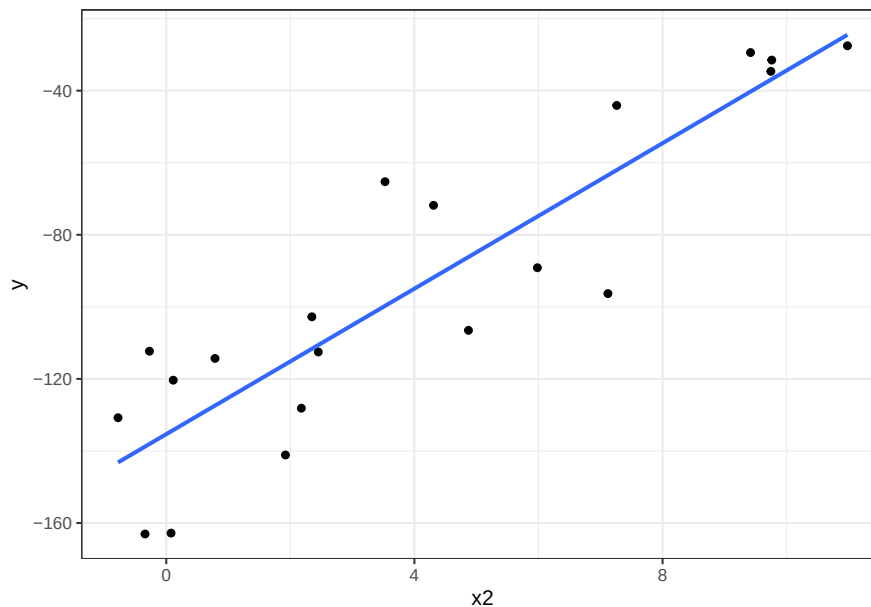
```
m2 <- lm(y ~ x2, d2)
summary(m2)
```

```
##
## Call:
```

```
## lm(formula = y ~ x2, data = d2)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -32.863 -16.174   3.808  13.270  34.473
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  -135.341     6.622  -20.437 6.63e-14 ***
## x2             10.095     1.194   8.452 1.11e-07 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 20.1 on 18 degrees of freedom
## Multiple R-squared:  0.7987, Adjusted R-squared:  0.7875
## F-statistic: 71.43 on 1 and 18 DF,  p-value: 1.111e-07
```

```
ggplot(d2, aes(x2, y)) +
  geom_point() +
  geom_smooth(method = 'lm', se = FALSE)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



However, that does not give us a full picture since the value of y now depends on both x_1 and x_2 . Given that these effects are independent (i.e. the effect of x_1 does not depend on the value of x_2 , and vice versa), we can ask the following question: what is the effect of one of the predictors when the value of the other is held constant? In that way, we can investigate the effect of an individual predictor while controlling for the effect of the others.

This is exactly what multiple regression lets us do. The way to build such a model is very similar to simple regression, we just specify all predictors on the right-hand side of the regression formula and separate them with +:

```
m3 <- lm(y ~ x1 + x2, d2)
summary(m3)
```

```
##
```

```
## Call:
## lm(formula = y ~ x1 + x2, data = d2)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.5601 -0.4775  0.1934  0.6887  1.6912
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept) -5.25923     1.68401  -3.123  0.00619 **
## x1          -5.85429     0.07408 -79.021 < 2e-16 ***
## x2           7.04660     0.07476  94.257 < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.077 on 17 degrees of freedom
## Multiple R-squared:  0.9995, Adjusted R-squared:  0.9994
## F-statistic: 1.555e+04 on 2 and 17 DF,  p-value: < 2.2e-16
```

How do we interpret the coefficients of this model?

- Intercept: the value of y when both x_1 and x_2 are 0 equals -5.26.
- x_1 : the change in y corresponding to an increase of 1 in x_1 **while holding x_2 constant** equals -5.85.
- x_2 : the change in y corresponding to an increase of 1 in x_2 **while holding x_1 constant** equals 7.05.

Another thing to notice is that the coefficient of determination (R^2) in this model is higher than in either of the simple models:

```
summary(m1)$r.squared
```

```
## [1] 0.7138627
```

```
summary(m2)$r.squared
```

```
## [1] 0.7987269
```

```
summary(m3)$r.squared
```

```
## [1] 0.9994535
```

Exercise

Let's continue with our old friend, the `beginningReaders` data set:

```
library('languageR')
data('beginningReaders')
```

This time we will group the data by `Word` and get a summary with each word's frequency, length (in characters) and mean reaction time.

```
words <- beginningReaders %>%
  group_by(Word) %>%
  summarise(Freq = first(LogFrequency),
            OrthLength = first(OrthLength),
            RT = mean(LogRT))

head(words)
```

```
## # A tibble: 6 x 4
```

```
##   Word      Freq OrthLength   RT
##   <fct>    <dbl>      <int> <dbl>
## 1 avontuur  4.39         8  7.25
## 2 baden     2.64         5  7.40
## 3 balkon    3.74         6  7.37
## 4 band      5.03         4  7.26
## 5 barsten   2.94         7  7.47
## 6 beek      4.55         4  7.26
```

We'll start with a simple model predicting reaction time from word frequency:

```
words_m1 <- lm(RT ~ Freq, words)
summary(words_m1)
```

```
##
## Call:
## lm(formula = RT ~ Freq, data = words)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.32448 -0.06956 -0.00079  0.07334  0.27760
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  7.549531   0.036778  205.28 < 2e-16 ***
## Freq        -0.053461   0.008664   -6.17 4.31e-09 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.1105 on 182 degrees of freedom
## Multiple R-squared:  0.173, Adjusted R-squared:  0.1685
## F-statistic: 38.07 on 1 and 182 DF, p-value: 4.314e-09
```

What is the interpretation of coefficients in this model?

As we can see from the R^2 value, the model accounts for around 17% of variance in the data. In other words, there is plenty of unexplained variation left.

Create a model which **also** uses word length as a predictor. Compare the R^2 values and interpret model coefficients.

```
# Write your solution here.
```

Interpreting non-standardised coefficients

Once we introduce predictors expressed on different scales, interpreting sizes of individual effects will be difficult since it will depend on the range of observed values. If we divide x_2 by 10, its effect will suddenly seem much larger than that of x_1 :

```
d2 <- d2 %>%
  mutate(x2_dec = x2 / 10)
m4 <- lm(y ~ x1 + x2_dec, d2)
coefficients(m4)
```

```
## (Intercept)          x1          x2_dec
##  -5.259234    -5.854291    70.466044
```

As you remember, the solution is to standardise predictors, effectively expressing them in units of standard deviation:

```
d2 <- d2 %>%
  mutate(x1_z = (x1 - mean(x1)) / sd(x1),
         x2_z = (x2 - mean(x2)) / sd(x2))

m5 <- lm(y ~ x1_z + x2_z, d2)
summary(m5)

##
## Call:
## lm(formula = y ~ x1_z + x2_z, data = d2)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -2.5601 -0.4775  0.1934  0.6887  1.6912
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  -94.2259     0.2409  -391.10  <2e-16 ***
## x1_z          -22.8022     0.2886   -79.02  <2e-16 ***
## x2_z           27.1986     0.2886    94.26  <2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 1.077 on 17 degrees of freedom
## Multiple R-squared:  0.9995, Adjusted R-squared:  0.9994
## F-statistic: 1.555e+04 on 2 and 17 DF,  p-value: < 2.2e-16
```

Notice that since we have only used linear transformations, the value of R^2 in models m3, m4 and m5 is identical.

Exercise

Standardise predictors in the `beginningReaders` model and interpret its coefficients.

```
# Write your solution here.
```

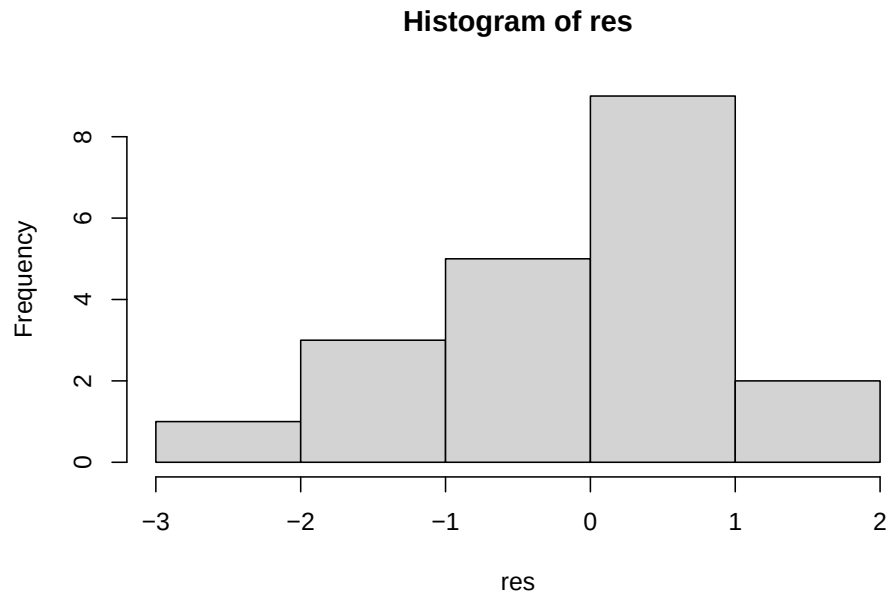
Model diagnostics

Fitting a model is easy but how do we know our model makes sense? On the one hand, the model makes sense when it is related to our research question. On the other hand, it also needs to satisfy the assumptions that all regression models make. These are largely related to distribution of residuals. As you remember, we can extract residuals from the model with the `residuals` function:

```
res <- residuals(m5)
```

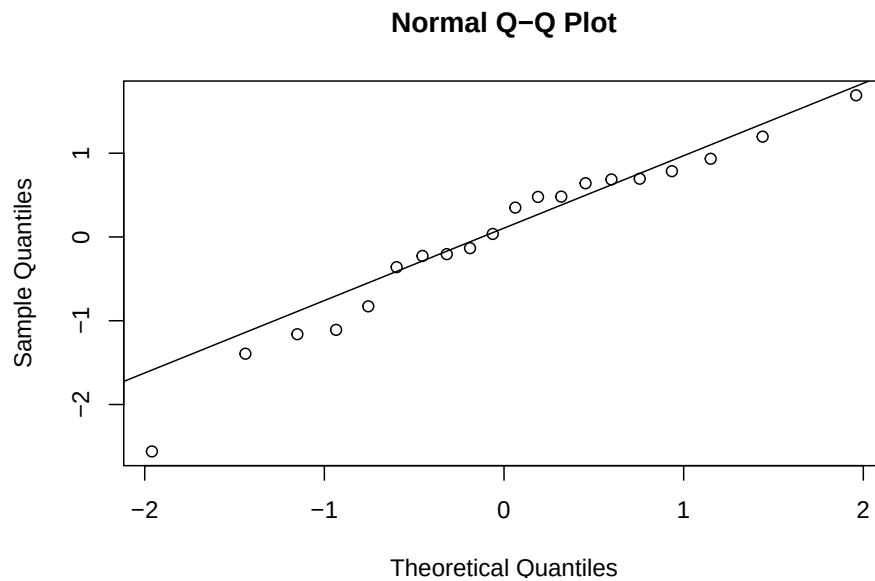
One important assumption is that residuals are supposed to be distributed normally. We can determine that by looking at their histogram:

```
hist(res)
```



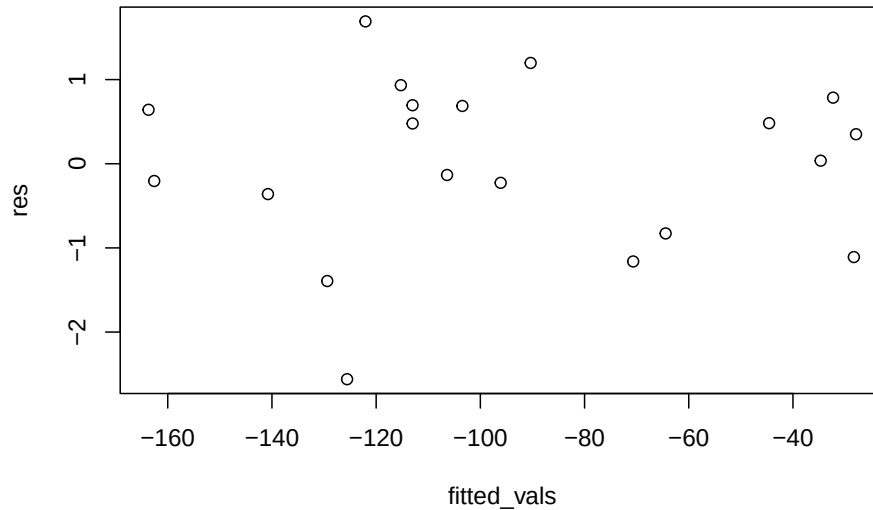
Another way is to compare the observed quantiles with quantiles of the normal distribution. In the perfect case, all data points should lie on the diagonal line:

```
qqnorm(res)
qqline(res)
```



The second important assumption concerns *homoscedacity* or constant variance irrespective of the fitted (i.e. predicted) values. In other words, when we plot residuals against the fitted values, we should see no obvious patterns – residuals should be distributed pretty much randomly. We can obtain the fitted values using the `fitted` function:

```
fitted_vals <- fitted(m5)
plot(fitted_vals, res)
```



Exercise

Repeat the steps above for the `beginning_readers` model. Are any of the regression assumptions violated in this case?

Write your solution here.

Independence assumption

Another important assumption of linear regression models is that the data points need to be *independent* from each other. Let's consider the original `beginningReaders` data set again. We might be tempted to fit the model this data directly but there is one problem – since each participant in the experiment rated all the words, we have many reaction time values per participant. We can get the actual numbers using `table()`:

```
table(beginningReaders$Subject)
```

```
##
## S10 S12 S13 S14 S15 S16 S18 S2 S20 S21 S22 S26 S27 S28 S29 S3 S30 S31 S32 S33
## 160 174 153 118 165 94 110 132 114 163 117 158 118 149 97 142 99 138 136 114
## S34 S35 S37 S39 S40 S43 S46 S47 S48 S49 S51 S52 S53 S54 S57 S58 S59 S60 S61 S62
## 105 153 109 117 126 128 180 150 123 102 164 177 164 158 146 147 141 126 166 137
## S63 S64 S65 S66 S67 S68 S69 S7 S70 S71 S72 S73 S74 S75 S76 S77 S78 S79 S8
## 174 103 169 71 120 127 135 120 138 156 144 128 172 171 124 91 130 84 96
```

Clearly, data points from a single child are not independent and this will cause our model to be too confident about its own predictions. The way I have dealt with it here was to calculate mean reaction times per word or participant, which is not optimal since we reduce our data and lose useful information in the process. A more elegant solution to this problem is hierarchical (or mixed-effects) models, which allow you to specify to use the full data and instead tell the model what kind of interdependencies are present and how your data should be grouped. This material is covered in chapters 14 and 15 of the textbook.

Correlated predictors

It is often the case that the predictors we are interested in are correlated. Below we generate correlated variables using the function `generate_correlated` defined in the file `06_utils.R`. It takes two arguments: the correlation coefficient (`rho`) and sample size (`n`)

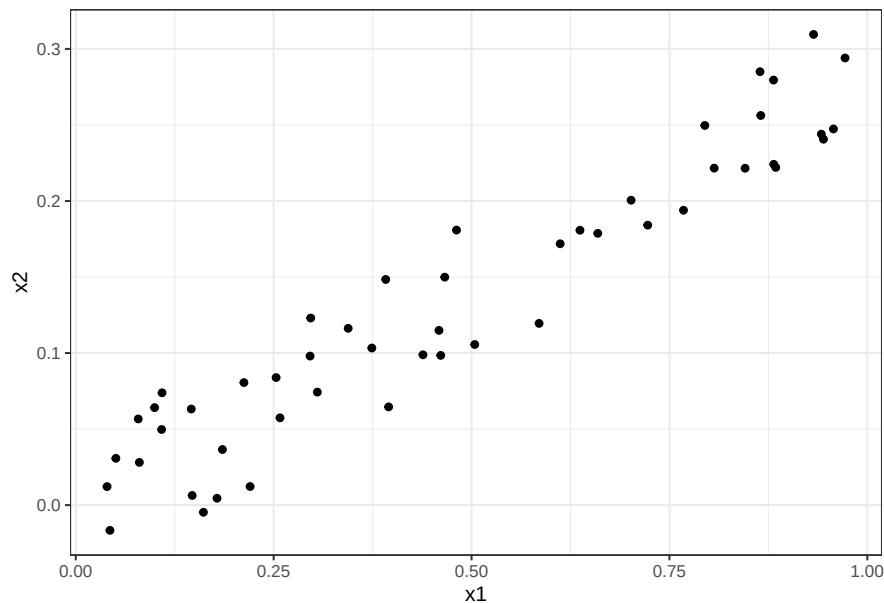

```
d_cor = generate_correlated(rho = 0.95, n = 50)
```

Let's if it worked:

```
cor(d_cor$x1, d_cor$x2)
```

```
## [1] 0.95
```

```
ggplot(d_cor, aes(x1, x2)) +  
  geom_point()
```



When two variables are correlated, their values co-vary, that is change the value of one changes the other. This is clearly a problem since we said above that regression tries to estimate the effect of changing one predictor *while keeping the other ones constant*. Thus, when predictors are very strongly correlated, our models will have a hard time estimating individual (partial) effects.

Let's see this in action by generating a dependent variable using the correlated predictors:

```
d_cor <- d_cor %>%  
  mutate(y = 3 + 2 * x1 - 6 * x2 + rnorm(nrow(d_cor), 0, 1))
```

We can now fit a simple regression model with one predictor:

```
m_cor1 <- lm(y ~ x1, d_cor)  
coefficients(m_cor1)
```

```
## (Intercept)          x1  
##   2.7384123    0.6177673
```

Everything looks fine so far. But what happens when we add x2 to the mix?

```
m_cor1 <- lm(y ~ x1 + x2, d_cor)  
coefficients(m_cor1)
```

```
## (Intercept)          x1          x2  
##   2.738412    6.006010  -19.347839
```

You can see that the coefficient values have changed quite a bit now. This is exactly because `lm()` cannot estimate individual effects.

Another consequence of including correlated predictors is that small changes to the data can produce large differences in parameter estimates. Below we use `slice_sample` to create a random sample of 80% of rows from the `d_cor` data set and use that to refit the model. If you execute the following chunk a couple of time, you should be able to see coefficients estimates vary subsequently across runs.

```
d_cor1 <- d_cor %>%  
  slice_sample(prop = 0.8)  
coef(lm(y ~ x1 + x2, d_cor1))
```

```
## (Intercept)          x1          x2  
##    2.385034    5.985187   -17.491900
```

Adjusted R^2

We have seen that adding predictors increased the R^2 value. But what happens if we add a random predictor unrelated to our outcome variable?

```
d2 <- d2 %>%  
  mutate(x3 = rnorm(nrow(d2)))  
m6 <- lm(y ~ x1 + x3, d2)
```

Compared to the model with `m1` as the only predictor, R^2 went up slightly:

```
summary(m1)$r.squared
```

```
## [1] 0.7138627
```

```
summary(m6)$r.squared
```

```
## [1] 0.7140058
```

This is generally the case: adding extra variables will result in a increase of R^2 . However, we want our models to be *parsimonious*, that is we want to achieve a good fit using only a few relevant and strong predictors. To penalize models which use too many variables, we can use adjusted R^2 instead:

```
summary(m1)$adj.r.squared
```

```
## [1] 0.6979662
```

```
summary(m6)$adj.r.squared
```

```
## [1] 0.6803594
```

Exercise

Let's add a new column with some randomly generated to the `words` data:

```
words <- words %>%  
  mutate(Rand = runif(nrow(words)))
```

Refit the model using this data as an additional predictor. Compare the R^2 and the adjusted R^2 values:

```
# Write your solution here.
```